

Special Participation E No 2

Using Gemini:

Me:

write me an LLM prompt to get good test questions from topics from this homework

Included the PDF of hw 12

Gemini:

Here is a comprehensive prompt you can use. I have structured it to ensure the LLM acts as a graduate-level Teaching Assistant and covers the specific nuances found in your homework file (like the specific Transformer initialization bug and the Forward vs. Reverse KL distinction).

You can copy and paste the block below directly into an LLM.

Prompt:

You are an expert Teaching Assistant for a graduate-level Deep Learning course (specifically EECS 182). I need you to generate a set of practice exam questions based on the concepts covered in "Homework 12," which focuses on Transformers, Variational methods, and Meta-Learning.

Context & Source Material:

Please generate questions based on the following specific topics and constraints derived from the homework:

1. Debugging Transformers:

- Focus on weight initialization issues in Encoder-Decoder architectures (specifically the logic of sharing weights between the embedding layer and the language model head).

- Question style: Debugging code snippets or explaining why loss curves might not converge.

2. KL Divergence (Forward vs. Reverse):

- Concepts: The asymmetry of $D_{KL}(P||Q)$ vs $D_{KL}(Q||P)$.
- Focus: Which direction results in "mode-seeking" vs. "mean-seeking" behavior (covering the entire distribution vs. fitting a specific mode).
- Scenario: Fitting a Gaussian Q to a bimodal distribution P .

3. Variational Information Bottleneck (VIB):

- Concepts: Encoder-Decoder structures, latent variables Z , maximizing task utility while minimizing information about the input (regularization).
- Math: The tradeoff parameter β in the loss function $L = \text{Task Loss} + \beta \cdot D_{KL}$.
- Analysis: How changing β affects the latent space embeddings (e.g., clustering of classes vs. collapse to the prior).

4. Variational Autoencoders (VAEs):

- Concepts: The Evidence Lower Bound (ELBO), the Reconstruction Term vs. the KL Prior matching term.
- Implementation: The "Reparameterization Trick" ($z = \mu + \sigma \odot \epsilon$) and why it is necessary for backpropagation.

5. Meta-Learning (MAML):

- Concepts: "Fast weights" (inner loop/adaptation) vs. "Slow weights" (outer loop/evolution).
- Algorithm: Model-Agnostic Meta-Learning (MAML).
- Scenario: Few-shot regression or classification.
- Theory: Why we compute gradients *through* the inner loop updates (meta-gradients).

6. Fine-tuning & Foundation Models (Bonus Topic):

- Concepts: Soft prompts, Low-Rank Adapters (LoRA), and Catastrophic Forgetting.

Instructions for Output:

Please generate 3 distinct types of questions for the topics above:

1. **Conceptual/Intuition Questions:** (e.g., "Why does Reverse-KL lead to mode-dropping behavior?")
2. **Mathematical/Derivation Questions:** (e.g., "Write out the ELBO objective and explain the role of the reconstruction term.")

3. **Scenario/Applied Questions:** (e.g., "If your VAE generates blurry images, which term in the loss function is likely dominating, and how would you adjust the hyperparameters?")

Format:

- Provide the question first.
 - Provide a detailed solution and explanation immediately after each question.
 - Keep the difficulty level suitable for a graduate computer science student.
-

Explanation of why this prompt works:

- **Role definition:** It forces the LLM to adopt the persona of a TA for EECS 182, ensuring the tone is academic and rigorous.
- **Specific Anchoring:** Instead of just asking for "Meta-learning questions," it references "Fast vs. Slow weights" and "Inner vs. Outer loop," which are specific framings used in your PDF1.
- **Visual Interpretation translation:** The prompt explicitly mentions the "Forward vs. Reverse KL" visual problem from the homework2, ensuring the LLM generates questions about mode-seeking behavior which was a key part of the assignment.
- **Code Context:** It flags the specific "weight tying" bug in Transformers3, which was the core debugging task in the homework.

Gemini Study materials:

Hello there! As a TA for EECS 182, I'd be happy to help you prepare for the exam. It looks like you're diving deep into the generative and meta-learning modules of the course—these are definitely some of the most conceptually dense topics we cover!

Here is a set of practice questions designed to test your understanding of "Homework 12" topics, structured as you requested with the solution immediately following.

Topic 1: KL Divergence & Variational Inference

Type: Conceptual / Intuition

Question 1:

Consider a target distribution $P(x)$ that is bimodal (has two distinct peaks). We wish to approximate P using a unimodal Gaussian distribution $Q_\theta(x)$ by minimizing the Kullback-Leibler divergence.

Compare the qualitative results of optimizing these two different objectives:

1. **Forward KL:** $\min_\theta D_{KL}(P||Q_\theta)$
2. **Reverse KL:** $\min_\theta D_{KL}(Q_\theta||P)$

Explain which objective leads to "mean-seeking" behavior and which leads to "mode-seeking" behavior. Why does this happen?

Solution:

- **Forward KL** ($D_{KL}(P||Q) = \mathbb{E}_{x \sim P}[\log P(x) - \log Q(x)]$): This objective leads to **mean-seeking** (or zero-avoiding) behavior.
 - **Reasoning:** To minimize the expectation, wherever $P(x) > 0$, $Q(x)$ must also be non-zero (to avoid $-\log Q(x) \rightarrow \infty$). Consequently, Q will stretch to cover **both** modes of P , placing its mean roughly between them. This often results in Q having high variance and assigning probability mass to regions where P has low probability (the valley between modes).
 - **Reverse KL** ($D_{KL}(Q||P) = \mathbb{E}_{x \sim Q}[\log Q(x) - \log P(x)]$): This objective leads to **mode-seeking** (or zero-forcing) behavior.
 - **Reasoning:** Here, the expectation is taken over Q . If $P(x)$ is near zero, $Q(x)$ **must** be zero to minimize the penalty (since $-\log P(x)$ would be large). Therefore, Q will "squeeze" itself under one of the single modes of P to avoid areas where P is low, ignoring the other mode entirely.
-

Topic 2: Variational Autoencoders (VAEs)

Type: Mathematical / Derivation

Question 2:

In a VAE, we use the Reparameterization Trick to sample the latent variable z .

1. Write the standard expression for sampling z from the approximate posterior $q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \text{diag}(\sigma_\phi^2(x)))$ using this trick.
2. Mathematically, why is this trick necessary for the backpropagation algorithm to function efficiently? Specifically, what would go wrong with the gradient calculation $\nabla_\phi L$ if we simply sampled $z \sim \mathcal{N}(\mu_\phi, \sigma_\phi^2)$ directly?

Solution:

1. Expression:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$$

where $\epsilon \sim \mathcal{N}(0, I)$ is an auxiliary noise variable and $\odot$ denotes element-wise multiplication.

2. Necessity for Backpropagation:

- If we sample $z \sim q_\phi(z|x)$ directly, the operation is stochastic and "breaks" the computation graph. The node z would effectively be a random number generator node, and we cannot compute the derivative of a random sample with respect to the distribution parameters ϕ directly (the "score function" estimator is an alternative, but it has high variance).
- By rewriting z as a deterministic function of ϕ and a fixed noise source ϵ , we externalize the randomness. Now, z is differentiable with respect to μ and σ . This allows gradients to flow from the loss function, through z , back to the encoder outputs μ and σ via the chain rule:

$$\frac{\partial L}{\partial \phi} = \frac{\partial L}{\partial z} \left(\frac{\partial z}{\partial \mu} \frac{\partial \mu}{\partial \phi} + \frac{\partial z}{\partial \sigma} \frac{\partial \sigma}{\partial \phi} \right)$$

Topic 3: Variational Information Bottleneck (VIB)

Type: Scenario / Applied

Question 3:

You are training a classifier using the Variational Information Bottleneck method. Your loss function is:

$$L = \mathcal{L}_{Task}(Y, \hat{Y}) + \beta \cdot D_{KL}(E(Z|X) || P(Z))$$

You observe that your model has excellent accuracy on the training data, but the latent space visualizations show that the class clusters are overlapping significantly and the distinctness of the classes in Z -space is poor. Conversely, a colleague training with a different β finds their latent space has very sharp, distinct clusters, but their task accuracy is lower.

1. Is your β value likely higher or lower than your colleague's?
2. Explain how the value of β controls the trade-off between "compression" and "prediction."

Solution:

1. **Comparison:** Your β is likely **lower** than your colleague's.
 2. **Explanation:**
 - The term $\beta \cdot D_{KL}$ acts as a regularizer that forces the posterior $E(Z|X)$ to "forget" information about the input X by compressing it toward the prior $P(Z)$ (usually a standard Gaussian).
 - **High β :** Forces stronger compression (bottleneck). The model discards more information about X . If β is too high, the latent representation collapses to the prior (Posterior Collapse), effectively ignoring the input, which hurts task accuracy but makes Z very smooth/compact.
 - **Low β :** Weakens the bottleneck. The model is allowed to encode more information from X into Z to satisfy the task loss. This usually leads to better accuracy but potentially less "disentangled" or structured latent spaces, and a higher risk of overfitting to nuisance variables in X .
-

Topic 4: Debugging Transformers

Type: Scenario / Applied (Debugging)

Question 4:

You are implementing a Transformer for a sequence-to-sequence translation task. Following standard advice, you decide to tie the weights of the input embedding layer and the final output projection (language model head) layer.

After implementation, you notice your training loss is extremely high at the very start and does not converge, whereas it worked fine when the layers were separate.

You examine the code and see you initialized the embedding weights W_{emb} using a standard Xavier/Glorot initialization, and you used the exact same weight matrix for the output logits: $\text{logits} = h @ W_{\{emb\}}.T$.

What is the likely scale issue causing this instability, and how should you adjust the computation of the logits or the embeddings to fix it?

Solution:

- **The Issue:** In the Transformer architecture (specifically the formulation in *Attention Is All You Need*), the embedding vectors are often multiplied by $\sqrt{d_{model}}$ before being added to the positional encodings. This scales the magnitude of the vectors up.
- However, the output softmax layer expects logits that are roughly unit scale to produce a smooth distribution. If you reuse the unmodified embedding matrix W_{emb} (which might have small values from Xavier init) or the scaled embeddings directly without normalization, the dot products at the final layer might be inconsistent with the scale required for the softmax.
- **Likely Cause:** The specific bug often arises because the gradients for the shared matrix are summed from both the input (embedding) and output (head) roles. If the internal representations h grow large (due to LayerNorm and residual connections preserving variance) and are projected by a weight matrix optimized for input embedding scale, the resulting logits can be wildly off-scale.
- **Fix:** A common practice when tying weights is to rescale the weights in the output projection. For example, if the embeddings were scaled by $\sqrt{d_{model}}$ at the input, you might need to apply the reverse scaling or ensure `logits = (h @  $W_{\{emb\}}.T$ ) / sqrt(d_model)` (or similar variance preserving factor) so the softmax receives values with appropriate variance. Alternatively, ensure the shared W is unit-variance and handle the $\sqrt{d_{model}}$ multiplier explicitly only at the input addition step.

Topic 5: Meta-Learning (MAML)

Type: Conceptual / Theory

Question 5:

Model-Agnostic Meta-Learning (MAML) is often described as optimizing for parameters that are "easy to fine-tune."

1. Distinguish between the "**Fast Weights**" and "**Slow Weights**" in the context of the MAML algorithm.
2. Why does MAML require computing a Hessian-vector product (or a second-order derivative) in its standard form, and what does the "First-Order MAML" (FOMAML) approximation ignore?

Solution:

1. Fast vs. Slow Weights:

- **Slow Weights (θ):** These are the meta-parameters (the initialization) learned in the **outer loop**. They persist across different tasks and evolve slowly to find a good starting point.
- **Fast Weights (θ'_i):** These are the task-specific parameters computed in the **inner loop**. For a specific task $\mathcal{T}_i$, we adapt the slow weights via one (or a few) gradient steps (e.g., $\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(\theta)$). These weights are temporary and specific to that task.

2. Derivatives:

- **Second-Order Requirement:** The meta-objective maximizes the performance of the **adapted** model. We optimize $\mathcal{L}(\theta'_i)$. Since θ'_i is itself a function of the gradient of θ (i.e., $\theta - \alpha \nabla \mathcal{L}(\theta)$), the meta-gradient requires differentiating **through** the gradient operator: $\nabla_{\theta}[\mathcal{L}(\theta - \alpha \nabla_{\theta} \mathcal{L}(\theta))]$. By chain rule, this involves the gradient of a gradient, which is the Hessian (H).
 - **FOMAML:** First-Order MAML ignores the second-derivative term. It assumes the derivative of the inner-loop update with respect to the initialization is essentially Identity (ignoring the Hessian part). It essentially says, "move the initialization θ in the direction that the adapted weights θ'_i want to move on the validation set."
-

Hope these help you study! If you get stuck on the math for the ELBO derivation or need more details on LoRA for the bonus section, just let me know. Good luck!

Gemini Quiz:

The screenshot shows the Gemini AI web interface. On the left, a sidebar contains a menu icon, a share icon, and a search bar with the text "on the topic I was talking about before". Below this is a section titled "Analyzing the Request" with a dropdown arrow. The main content area on the left displays a practice quiz titled "Deep Learning: Transformers & Variational Methods" with a subtitle "Here is a practice quiz covering the Deep Learning topics we discussed, including Transformers, Variational Methods (VAEs, VIB), and Meta-Learning (MAML)." and a timestamp "Dec 9, 3:00 AM". A button "Try again without interactive quiz" is visible. At the bottom of the sidebar is a search bar with the placeholder text "What do you want to learn?". The main content area on the right is titled "Deep Learning Practice Exam Questions" and shows a progress bar at the top with "1 / 5" and "0" correct answers. The first question is: "1. In the context of approximating a bimodal target distribution $P(x)$ with a unimodal Gaussian $Q_\theta(x)$, minimizing the Reverse KL Divergence $D_{KL}(Q_\theta||P)$ typically results in which behavior?". The options are: A. High variance behavior (flattening the distribution), B. Perfect convergence to the mean of $P(x)$, C. Mean-seeking behavior (covering both modes), and D. Mode-seeking behavior (fitting a single mode). A "Show hint" button is below the options. A "Next" button is at the bottom right of the quiz area. The bottom of the interface shows a status bar with "Gemini can make mistakes, so double-check it".

Using youlearn

app.youlearn.ai/learn/content/dl6CSrwHcP5F7Xm

[Home](#)
[Learn](#)
[Exams](#)
[Quizzes](#)
[Flashcards](#)
[Podcast](#)
[Summary](#)
[Chapters](#)
[Notes](#)

Homework 12

Steps

2 / 12

Page fit

🔍

🔄

📄

(a) Your model's loss - 23.4

Figure 1: Comparing your model's loss vs your friend's model's loss. Your model is doing significantly worse.

Your friend suggests that there's something wrong with how the head gets initialized. **Identify the bug in your initialization, fix it by correcting the buggy lines, and briefly explain why your fix should work.**

Hint: remember that `d_model` is large in transformer models.

Hint: your change needs to impact line 9 somehow since that is where the head is initialized.

The bug: (brief description)

The fix (code): (Just show anything you change and/or add to the code.)

Why the fix should work: (brief explanation)

(b) Your friend's model's loss - 6.1

2. Comparing Distributions

Divergence metrics provide a principled measure of difference between a pair of distributions (P, Q). One such example is the Kullback-Leibler Divergence, that is defined as

$$D_{KL}(P||Q) = \mathbb{E}_{x \sim P(x)} \left[\log \frac{P(x)}{Q(x)} \right]$$

(a) Technically D_{KL} is not a true distance since it is asymmetric, i.e. generally $D_{KL}(P||Q) \neq D_{KL}(Q||P)$. **Give an example of univariate distributions P and Q where $D_{KL}(P||Q) \neq \infty, D_{KL}(Q||P) = \infty$.**

(b) For a fixed target distribution P, we call $D_{KL}(P||Q)$ the *forward-KL*, while calling $D_{KL}(Q||P)$ the *reverse-KL*. Due to the asymmetric nature of KL, distributions Q that minimize $D_{KL}(P||Q)$ can be different from those minimizing $D_{KL}(Q||P)$.

From the following plots, **identify which of (A, B) correspond to minimizing forward vs. reverse KL. Give brief reasoning.** Here, only the mean and standard deviation of Q is allowed to vary during the minimization.

A

B

Chat
Flashcards
Quizzes
Podcast
Summary
Chapters
Notes

Create me a @Quiz on question 2

Generated quiz ^

Quiz generated!

1. What makes KL divergence not a true distance metric?

A. It is not bounded: $D_{KL}(P||Q)$ can take on infinite values depending on the distributions

B. It is not defined for all probability distributions: $D_{KL}(P||Q)$ requires both P and Q to be valid

C. It is asymmetric: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$ in general

D. It does not satisfy the triangle inequality: $D_{KL}(P||R) \leq D_{KL}(P||Q) + D_{KL}(Q||R)$

🔄

✓

1 / 8

Don't know

Quiz on **Question 2: Comparing Distributions (KL Divergence)** generated and displayed above.

🔍

👍

👎

⋮

Learn anything

Auto
🔍 Add Context

🔍 📄 🗑️ 🔄

Voice

Homework 12, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission.
2

The screenshot shows a web application interface. On the left, a document titled "Generative Models and Variational Autoencoders" is displayed, showing a transcript of a conversation. On the right, a summary of the document is shown, including sections on "Post-Training: Supervised Fine-Tuning (SFT) and Beyond" and "Test-Time Compute Scaling".

Document Content (Left):

A lot of people are busily working on their project. So I'm gonna unfortunate but well deadlines have to be sometime. So um I will start with a couple of announcements. Uh I didn't announce this uh last time, but we we told people to fill out the survey and we said that we'll give extra credit if for everybody in the class, 3% uh if 75% of the class does the survey. I forgot to give a deadline. I put it on online. So this is due Friday of Thanksgiving week.

And uh the other uh important uh reminder is today your draft reports are due. We will quickly turn around and assign you a report to review uh using the conference management system. So you have can do that over the weekend and sort of by uh Tuesday before everyone goes off Thanksgiving break submit read the report of that you're assigned and write a review. This is important to give everyone feedback so that they can make their projects better. Uh there were some questions about when this uh report is due. So it's uh due uh tonight at the midnight. I think we will maybe accept some submissions if they come in late, but tomorrow morning we're going to be assigning people to a review so that we have to have it by then. Um, okay. So, that's the the expectation we have for everyone is that you've worked very hard. You're working on your reports and you are finding that there's a lot more you want to do than you have already done and that is to be expected. But you have to write up what you've done and what you're planning to do more based on what you've seen so that someone can give you feedback usefully.

Summary Content (Right):

Significance: Enables massive models on laptops/GPUs.

Post-Training: Supervised Fine-Tuning (SFT) and Beyond

Pretrain on next-token prediction; SFT fine-tunes on instruction-answer pairs (e.g., QA, no question generation).

- Chain-of-thought (CoT):** Prompts like "think step by step" elicit reasoning (from SFT data); boosts via more tokens/compute.
- Mid-training:** Mix synthetic CoT-like data into pretraining for better reasoning.

Test-Time Compute Scaling

Spend more inference compute for better answers (e.g., CoT). Key for reasoning models like o1 (uses RLVR-like).

- Pure prompting:** "Think step-by-step, check answers."
- Best-of-n:** Generate n samples; select (training: % correct; inference: majority vote, LLM grader, consensus).
- High-prob. selection:** Pick max log-prob. sequence (model self-grades via autoregressive probs.).
- Advanced sampling:** Raise dist. to power > 1 (tilting: amplifies high-prob. paths, shrinks low-prob. errors). Mimics RL-boosted high-prob. regions without training.
- Results:** On benchmarks, power-4 sampling rivals RLVR (e.g., 78% $\rightarrow$ near without tuning); transfers better.

Misconception: RL post-training learns new capabilities vs. just sharpening existing (open question; KL regularizer constrains shifts).

Connection: Sampling exploits autoregressive error accumulation; ties to beam search, MCMC for tilted dists.

Generative Models: Core Objective and Categories

Generative models aim to produce new data samples, unlike discriminative models focused on classification or regression, by learning to sample from an unknown data distribution using a parameterized form trained on data.

S

S

This involves feeding randomness into the model to generate novel examples.

S

- Unconditional generation:** Samples from a standalone distribution using only randomness as input, producing new data without specific conditions—the primary focus for teaching core ideas. s s
- Conditional generation:** Samples from a conditional distribution given some input conditioning (e.g., a text prompt in language models or image generation tools) plus randomness, ensuring outputs align with the condition. s s Building

on unconditional generation, this is practically more important as it enables targeted creation like prompt-based images. s

- **Significance:** Understanding generative modeling allows creating realistic synthetic data, crucial for applications like image synthesis or text generation, where the challenge is both sampling effectively and learning the distribution from finite data. s s

💡 **Key Insight:** All generative techniques fundamentally require injecting randomness (e.g., noise) and transforming it into data-like outputs, as computers perform deterministic operations.

...

The Manifold Hypothesis and Failures of Early Approaches

Natural images lie on a low-dimensional **manifold** (a thin subset) within the high-dimensional space of all possible images, where most points are noise-like and unnatural.

...

This hypothesis explains why naive methods fail for generation.

Classifier-Based Generation Fails

Using a trained classifier (e.g., cat detector) for conditional generation involves sampling random noise and performing gradient ascent on the classifier logit to "move" toward cat-like images.

...

- **Mechanism:** Start from noise off the manifold; gradients push toward high-confidence regions. s
- **Why it fails:** Gradients lead to confidently classified but still noisy, unnatural images on-manifold, never reaching realistic cats—early attempts overlooked the need for small, guided steps toward the manifold. ...

- **Connection to later ideas:** Diffusion models succeed by enabling incremental steps onto the manifold. s s
- **Think of it like:** Trying to navigate a foggy mountain (manifold) from random plains using a distant compass (classifier)—you confidently "arrive" at a noisy ridge, not the peak. s

Standard Autoencoder Fails for Generation

A standard **autoencoder** (encoder compresses input XX to low-dimensional latent ZZ , decoder reconstructs $X^{\wedge}X^{\wedge}$; trained to minimize $\|X-X^{\wedge}\|X-X^{\wedge}\|$) exploits low-dimensional structure for compression.

...

- **Generation attempt:** Sample random ZZ from a simple prior (e.g., Gaussian) and decode. s s
- **Why it fails:** Encoder maps data to a thin low-dimensional subset of ZZ -space; random ZZ falls outside, causing neural nets to extrapolate poorly into blurry/noisy blobs, not cats. ...
- **Even with "right" dimension:** Performance improves gradually with larger ZZ , without a crisp optimum, yielding persistently bad samples. ...
- **Think of it like:** Encoder traces a thin thread in ZZ -space; random sampling hits unrelated voids, producing garbage—like extrapolating a map beyond explored territory. s s

⚠ **Warning:** Discrete dimensionality doesn't create a sharp bottleneck; VAEs use softer regularization via distributional losses.

s

s

Variational Autoencoders (VAEs): Fixing the Latent Space

VAEs (Variational Autoencoders) modify autoencoders to enable robust generation by making the encoder output a **distribution** over $\mathbb{Z}$ (not a point), matching a simple prior via regularization, ensuring decoder sees prior-like $\mathbb{Z}$ s during training.

s

s

This builds on autoencoders by adding randomness and distributional matching for generation readiness.

s

Encoder Outputs Parameters of a Distribution

- Encoder processes $\mathbf{x}$ to output **mean** $\boldsymbol{\mu}$ (vector) and parameters for **covariance** Σ , forming $q(\mathbf{z}|\mathbf{x}) \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$. s s
- Covariance parameterization**: To ensure positive semi-definite (required for valid Gaussian covariance), output a matrix $\mathbf{L}$ where $\Sigma = \mathbf{L}\mathbf{L}^T$ (Cholesky-like decomposition). ... Symmetry is automatic; this guarantees validity and allows gradients. s s
- Significance**: Avoids invalid matrices from direct output, enabling stable training. s s

KL Divergence: Regularizing to a Prior

KL divergence ($D_{KL}(Q\|P) = \mathbb{E}_Q[\log(Q(x)/P(x))]$) measures how much Q diverges from P , zero if identical.

...

- Asymmetry intuition**: Captures probabilistic distinguishability (e.g., fair vs. two-headed coin: fair coin can mimic two-headed briefly, but not vice versa). ... Place desired prior P (target) second, model dist. Q first to penalize Q producing low- P events. ...
- Choice for prior**: Standard isotropic Gaussian $\mathcal{N}(\mathbf{0}, \mathbf{I})$ (easy to sample, closed-form KL). s s

- **Closed-form KL for**

Gaussians: $DKL(N(\mu, \Sigma) \| N(0, I)) = \frac{1}{2}(\text{tr}(\Sigma) + \|\mu\|^2 - k - \log \det(\Sigma))$ $DKL(N(\mu, \Sigma) \| N(0, I)) = \frac{1}{2}(\text{tr}(\Sigma) + \|\mu\|^2 - k - \log \det(\Sigma))$ where k is dimension; computable directly from parameters. ...

- **Loss:** Reconstruction $\|X - X^{\wedge}\| + \beta DKL(q(Z|X) \| N(0, I))$ $\beta DKL(q(Z|X) \| N(0, I))$, with β balancing. $s \quad s_{_}$

Note: KL is not symmetric; variants exist, but this asymmetry aligns with generative needs.

s

s

Reparameterization Trick: Differentiable Sampling

To sample $Z \sim N(\mu, \Sigma)$ $Z \sim N(\mu, \Sigma)$ with gradients through randomness:

- Draw standard noise $v \sim N(0, I)$ $v \sim N(0, I)$.
- Compute $Z = \mu + Lv$ $Z = \mu + Lv$ (where $LL^T = \Sigma$ $LL^T = \Sigma$); linear transform yields desired Gaussian. ...
- **Gradients flow:** Backprop passes through addition/multiplication straight to μ, L, v (noise v treated as fixed input, like SGD batch stochasticity—expectation approximates true gradient). ...
- **Think of it like:** Noise v as data augmentation; gradients ignore it (go to zero mean), focusing on mean path, enabling end-to-end SGD. ...

Component	Standard Autoencoder	VAE
Encoder Output	Fixed Z	Dist. params. μ, Σ
Latent Sampling	None (deterministic)	$Z = \mu + Lv$, $v \sim N(0, I)$
Losses	Reconstruction only	Reconstruction + KL to prior
Generation	Random $Z \rightarrow$ poor	Prior $Z \sim N(0, I) \rightarrow$ better (blobby manifold)
Gradient Flow	Direct	Through reparam. trick
s		

Component	Standard Autoencoder	VAE
s		

Tension Between Losses and Information Bottleneck

- **Trade-off:** Reconstruction pushes high info through ZZ (precise $X^{\wedge}X^{\wedge}$); KL shrinks variance toward prior (low info, robust decoder). ...
- **$\beta\beta$ controls bottleneck:** High $\beta\beta \rightarrow$ low mutual info $I(X;Z)I(X;Z)$, worse recon. but prior-like ZZs (good generation); low $\beta\beta \rightarrow$ sharp recon., thin ZZ (poor generation). ... Continuous control vs. discrete dim. s
- **Effect in latent space:** Thickens thin manifold subset into "blobby" region, easing sampling (foreshadows latent diffusion). ...
- **Generation:** Sample $Z \sim N(0,I)Z \sim N(0,I)$, decode (ignores encoder). s s

💡 **Key Insight:** VAEs don't fully solve but mitigate extrapolation by training decoder on prior-like ZZs.

...

VAE Limitations and Tricks

- **Persistent tension:** High-fidelity recon. harms prior matching; tricks like iterative encode-decode or diffusion refine. ...
- **Why not train decoder on pure noise?:** Lacks info linking noise to specific XX (impossible multi-task). ...

Broader Tricks: Straight-Through Sampling and Quantization-Aware Training

VAE reparam. generalizes: Treat non-differentiable ops (sampling, quantization) as "additive noise," pass gradients straight through identity path.

...

- **Quantization-aware training:** For deploying large models on limited hardware (e.g., int4 vs. float32: 8x smaller, faster). s s

- Forward: Quantize weights (non-diff. rounding).
- Backward: Straight-through (treat as identity + noise; gradients bypass quant. to params). ...
- **Why it works**: Mimics inference; trains robustness to quantization noise. s
- **Significance**: Enables massive models on laptops/GPUs. s s

Post-Training: Supervised Fine-Tuning (SFT) and Beyond

Pretrain on next-token prediction; **SFT** fine-tunes on instruction-answer pairs (e.g., QA, no question generation).

s

s

- **Chain-of-thought (CoT)**: Prompts like "think step by step" elicit reasoning (from SFT data); boosts via more tokens/compute. s s
- **Mid-training**: Mix synthetic CoT-like data into pretraining for better reasoning. s

Test-Time Compute Scaling

Spend more inference compute for better answers (e.g., CoT).

s

s

Key for reasoning models like o1 (uses RLVR-like).

s

S

- **Pure prompting:** "Think step-by-step, check answers." s
- **Best-of-n:** Generate n samples; select (training: % correct; inference: majority vote, LLM grader, consensus). ...
 - **High-prob. selection:** Pick max log-prob. sequence (model self-grades via autoregressive probs.). s s
- **Advanced sampling:** Raise dist. to power >1 (tilting: amplifies high-prob. paths, shrinks low-prob. errors). s s Mimics RL-boosted high-prob. regions without training. s s
 - **Results:** On benchmarks, power-4 sampling rivals RLVR (e.g., 78% $\rightarrow$ near without tuning); transfers better. s s

⚠ **Misconception:** RL post-training learns new capabilities vs. just sharpening existing (open question; KL regularizer constrains shifts).

S

S

Connection: Sampling exploits autoregressive error accumulation; ties to beam search, MCMC for tilted dists.

...